ANSIBLE

Complete Reference Guide

Inventory • Ad-Hoc Commands • Playbooks • Modules

From basics to playbooks — everything you need to automate infrastructure with Ansible

Table of Contents

#	Chapter	Topics
1	**Introduction to Ansible**	What it is, what it does, when to use it
2	**Advantages & Disadvantages**	Pros and cons at a glance
3	**Inventory**	Static & dynamic inventory, file formats
4	**ansible.cfg — Configuration File**	Key directives and common settings
5	**Ad-Hoc Commands**	One-liners, modules, arguments
6	**Playbooks**	Structure, writing, running

Chapter 1 — Introduction to Ansible

What is Ansible?

Ansible is an open-source **IT automation tool** developed by Michael DeHaan and acquired by Red Hat in 2015. It automates **configuration management**, **application deployment**, **task automation**, and **multi-node orchestration** across servers, cloud environments, network devices, and containers.

Ansible is **agentless** — it communicates with managed nodes over SSH (Linux) or WinRM (Windows) without installing any software on them. It uses human-readable **YAML** files called *Playbooks* to define automation tasks.

Key Facts

- Language: Python (control node), YAML (playbooks)
- Protocol: SSH / WinRM — no agent needed on managed nodes
- Idempotent: running the same playbook multiple times produces the same result
- Open-source (Apache 2.0) with a commercial version: Red Hat Ansible Automation Platform

What is Ansible Used For?

- **Configuration Management** — ensure servers are always in a desired state (install packages, set configs)
- **Application Deployment** — deploy code and services consistently across environments
- **Provisioning** — spin up and configure cloud instances (AWS, Azure, GCP)
- **Orchestration** — coordinate rolling updates across multiple servers in the right order
- **Security & Compliance** — enforce hardening policies, manage users and SSH keys
- **CI/CD Pipelines** — integrate with Jenkins, GitLab CI, GitHub Actions
- **Network Automation** — manage routers, switches, firewalls (Cisco, Juniper, etc.)

When Should You Use Ansible?

Use Ansible when you need to:

- Manage **many servers** and want to avoid manual, repetitive work
- Ensure **consistency** — same config across dev, staging, and production
- Perform **zero-downtime rolling deployments**
- Quickly provision **cloud infrastructure** without writing complex scripts
- Integrate automation into **existing DevOps workflows** without heavy tooling
- Audit and enforce **compliance** across your fleet

Chapter 2 — Advantages & Disadvantages

■ Advantages	■ Disadvantages
✓ Agentless — no software on managed nodes	✓ Slower than agent-based tools (Puppet, Chef)
✓ Simple YAML syntax — easy to read/write	✓ No persistent state by default
✓ Idempotent — safe to run multiple times	✓ Complex logic can get messy in YAML
✓ Large built-in module library (6000+ modules)	✓ Debugging errors in large playbooks is hard
✓ SSH-based — uses existing infrastructure	✓ Windows support less mature than Linux
✓ Works on Linux, Windows, network devices	✓ Requires Python on managed nodes
✓ Strong community & Red Hat support	✓ No built-in GUI (AWX/Tower needed)
✓ Integrates with cloud providers natively	✓ Large inventories can be slow without tuning
✓ Low learning curve for sysadmins	✓ Variable precedence rules can be confusing
✓ Parallel execution across hosts	

Chapter 3 — Inventory

What is Inventory?

The **Ansible Inventory** is a file (or script) that lists all the managed nodes (hosts/servers) Ansible should work with. It is the foundation of any Ansible project — without an inventory, Ansible does not know which machines to connect to.

What is Inventory Used For?

- Define **which hosts** Ansible manages
- Group hosts logically (web servers, DB servers, regions, environments)
- Set **host-specific variables** (SSH user, port, Python path)
- Set **group variables** shared by multiple hosts
- Target specific groups in playbooks and ad-hoc commands

How to Create an Inventory

1. INI Format (default, most common)

```
# /etc/ansible/hosts OR ./inventory


[webservers]

web1.example.com

web2.example.com ansible_user=ubuntu ansible_port=2222


[dbservers]

db1.example.com

db2.example.com


[all:vars]

ansible_python_interpreter=/usr/bin/python3


[webservers:vars]
```

```
http_port=80

max_clients=200
```

2. YAML Format

```
# inventory.yml

all:

children:

webservers:

hosts:

web1.example.com:

web2.example.com:

ansible_user: ubuntu

vars:

http_port: 80

dbservers:

hosts:

db1.example.com:

db2.example.com:
```

3. Dynamic Inventory (Cloud)

For cloud environments, Ansible supports **dynamic inventory scripts** or plugins that query AWS, Azure, GCP, etc. automatically.

```
# Use AWS EC2 plugin

# inventory/aws_ec2.yml

plugin: amazon.aws.aws_ec2

regions:
```

```
  - us-east-1

  filters:

  instance-state-name: running

  keyed_groups:

  - key: tags.Role

  prefix: role
```

Useful Inventory Commands

- ansible-inventory --list # show full inventory as JSON
- ansible-inventory --graph # tree view of groups
- ansible all --list-hosts # list all hosts
- ansible webservers --list-hosts # list hosts in a group
- ansible-playbook -i inventory.yml # use custom inventory file

Chapter 4 — ansible.cfg Configuration File

What is ansible.cfg?

ansible.cfg is Ansible's main configuration file. It controls the default behavior of Ansible — where to find the inventory, which user to connect as, SSH settings, privilege escalation, output format, and much more.

File Search Order (highest priority first)

- **ANSIBLE_CONFIG** environment variable (if set)
- **./ansible.cfg** — in the current working directory ← most common
- **~/.ansible.cfg** — user's home directory
- **/etc/ansible/ansible.cfg** — system-wide default

Common ansible.cfg Settings

```
[defaults]

inventory = ./inventory # path to inventory

remote_user = ec2-user # SSH login user

private_key_file = ~/.ssh/id_rsa # SSH private key

host_key_checking = False # skip host key verification

forks = 10 # parallel connections

timeout = 30 # SSH timeout (seconds)

log_path = /var/log/ansible.log # enable logging

retry_files_enabled= False # don't create .retry files

stdout_callback = yaml # prettier output


[privilege_escalation]

become = True # enable sudo

become_method = sudo
```

```
become_user = root

become_ask_pass = False # don't prompt for sudo pw


[ssh_connection]

ssh_args = -o ControlMaster=auto -o ControlPersist=60s

pipelining = True # speed up execution

control_path = /tmp/ansible-ssh-%h-%p-%r
```

Key Directives Explained

- forks: how many hosts to manage in parallel (default 5)
- become: enable privilege escalation (sudo) globally
- host_key_checking: set False in lab/dev to avoid SSH prompt
- pipelining: reduces SSH operations — significant speed boost
- stdout_callback: 'yaml' or 'debug' gives much cleaner output
- retry_files_enabled: set False to avoid clutter

Chapter 5 — Ad-Hoc Commands

What are Ad-Hoc Commands?

Ad-hoc commands are one-line Ansible commands you run directly from the terminal without writing a Playbook. They are perfect for quick, one-time tasks — checking connectivity, restarting a service, or copying a file — when you don't need a reusable automation script.

What Are Ad-Hoc Commands Used For?

- Test connectivity to managed hosts (**ping**)
- Gather system facts (OS, CPU, memory)
- Install, remove, or update packages on the fly
- Start, stop, or restart services
- Execute shell commands across many hosts
- Copy files to/from remote hosts
- Manage users and groups
- Fetch information without writing a full playbook

General Syntax

```
ansible -m -a '' [options]


# Common options:

-i inventory_file # specify inventory

-u remote_user # SSH username

-b # become (sudo)

-k # ask for SSH password

-K # ask for sudo password

--become-user=root # sudo as this user
```

Built-in Modules & Examples

Module	Purpose	Example

ping	Test SSH connectivity	ansible all -m ping
command	Run a command (no shell)	ansible all -m command -a 'uptime'
shell	Run a shell command	ansible all -m shell -a 'df -h | tail -5'
copy	Copy file to remote	ansible web -m copy -a 'src=app.conf dest=/etc/app.c
fetch	Fetch file from remote	ansible db1 -m fetch -a 'src=/etc/my.cnf dest=./back
file	Manage files/dirs/perms	ansible all -m file -a 'path=/tmp/test state=touch m
apt	Manage deb packages	ansible all -b -m apt -a 'name=nginx state=present'
yum/dnf	Manage rpm packages	ansible all -b -m yum -a 'name=httpd state=latest'
service	Manage services	ansible all -b -m service -a 'name=nginx state=start
user	Manage users	ansible all -b -m user -a 'name=deploy state=present
setup	Gather host facts	ansible all -m setup
debug	Print variable/message	ansible all -m debug -a 'msg=hello'
uri	HTTP requests	ansible all -m uri -a 'url=http://localhost'
cron	Manage cron jobs	ansible all -b -m cron -a 'name=backup minute=0 hour

Practical Examples

Connectivity & Facts

```
# Ping all hosts

ansible all -m ping


# Ping only webservers group

ansible webservers -m ping


# Get all facts about a host

ansible web1 -m setup


# Filter specific facts

ansible all -m setup -a 'filter=ansible_distribution*'
```

Package Management

```
# Install nginx (Debian/Ubuntu)

ansible webservers -b -m apt -a 'name=nginx state=present update_cache=yes'


# Install multiple packages

ansible all -b -m apt -a 'name=git,curl,vim state=present'


# Remove a package

ansible all -b -m apt -a 'name=apache2 state=absent'


# Install on RHEL/CentOS

ansible all -b -m yum -a 'name=httpd state=latest'
```

Service Management

```
# Start a service

ansible webservers -b -m service -a 'name=nginx state=started'


# Restart a service

ansible webservers -b -m service -a 'name=nginx state=restarted'


# Enable service at boot

ansible webservers -b -m service -a 'name=nginx enabled=yes state=started'
```

File Operations

```
# Copy a local file to remote hosts

ansible all -m copy -a 'src=/local/file.conf dest=/remote/file.conf owner=root
mode=0644'


# Create a directory

ansible all -b -m file -a 'path=/opt/myapp state=directory mode=0755'
```

```
# Delete a file

ansible all -b -m file -a 'path=/tmp/oldfile state=absent'


# Fetch a file from remote

ansible db1 -m fetch -a 'src=/etc/mysql/my.cnf dest=./backups/ flat=yes'
```

Chapter 6 — Playbooks

What is a Playbook?

A **Playbook** is an Ansible configuration file written in **YAML** that defines a series of *plays*. Each play targets a group of hosts and runs a list of *tasks*. Tasks call Ansible modules to perform the actual work. Playbooks are reusable, version-controlled, and self-documenting.

When to Use a Playbook?

- When you need to run **multiple tasks in a specific order**
- When the automation should be **repeatable and version-controlled** (stored in Git)
- When you need **conditionals, loops, variables, and handlers**
- For **multi-tier deployments** (configure DB first, then web, then LB)
- When ad-hoc commands become too complex or long to type each time
- For **CI/CD pipelines** where automation must be scripted and reliable

Playbook Structure

```
# A playbook is a YAML list of 'plays'

---

- name: Play 1 — configure web servers

hosts: webservers # target group from inventory

become: yes # run as root (sudo)

vars: # play-level variables

http_port: 80

doc_root: /var/www/html


handlers: # triggered by 'notify'

- name: restart nginx

service:
```

```
name: nginx

state: restarted


tasks: # list of tasks (run in order)

- name: Install nginx

apt:

name: nginx

state: present

update_cache: yes


- name: Deploy nginx config

template:

src: nginx.conf.j2

dest: /etc/nginx/nginx.conf

notify: restart nginx # trigger handler on change


- name: Ensure nginx is running

service:

name: nginx

state: started

enabled: yes
```

Writing a Playbook — Step by Step

Step 1 — Start with --- (YAML document marker)

```
---
```

Step 2 — Define the play

```
- name: My first play

hosts: all # or a group name: webservers, dbservers

become: yes # optional: sudo
```

Step 3 — Add variables (optional)

```
vars:

app_name: myapp

app_port: 8080

packages:

- nginx

- git

- curl
```

Step 4 — Write tasks

```
tasks:

- name: Install required packages

apt:

name: '{{ packages }}'

state: present

update_cache: yes


- name: Create app directory

file:

path: /opt/{{ app_name }}

state: directory

mode: '0755'
```

```
- name: Copy app config
template:
src: app.conf.j2
dest: /etc/{{ app_name }}/app.conf
notify: restart app

- name: Conditional task (only on Ubuntu)
debug:
msg: 'This is Ubuntu {{ ansible_distribution_version }}'
when: ansible_distribution == 'Ubuntu'

- name: Loop example — create users
user:
name: '{{ item }}'
state: present
loop:
- alice
- bob
- carol
```

Step 5 — Add handlers

```
handlers:
- name: restart app
service:
name: '{{ app_name }}'
```

```
state: restarted
```

Complete Real-World Playbook Example

```
---

- name: Deploy NGINX web server

hosts: webservers

become: yes

vars:

nginx_port: 80

web_root: /var/www/html


handlers:

- name: reload nginx

service:

name: nginx

state: reloaded


tasks:

- name: Update apt cache

apt:

update_cache: yes

cache_valid_time: 3600


- name: Install nginx

apt:

name: nginx
```

```
    state: present

- name: Create web root
  file:
    path: '{{ web_root }}'
    state: directory
    owner: www-data
    group: www-data
    mode: '0755'

- name: Deploy index.html
  copy:
    content: 'Hello from Ansible!'
    dest: '{{ web_root }}/index.html'
  notify: reload nginx

- name: Start and enable nginx
  service:
    name: nginx
    state: started
    enabled: yes
```

How to Run a Playbook

```
# Basic run
ansible-playbook playbook.yml
```

```
# Specify inventory file

ansible-playbook -i inventory.ini playbook.yml


# Dry run (check mode — no changes made)

ansible-playbook playbook.yml --check


# Verbose output

ansible-playbook playbook.yml -v # verbose

ansible-playbook playbook.yml -vv # more verbose

ansible-playbook playbook.yml -vvv # connection debug


# Limit to specific hosts or groups

ansible-playbook playbook.yml --limit webservers

ansible-playbook playbook.yml --limit web1.example.com


# Run only specific tags

ansible-playbook playbook.yml --tags install

ansible-playbook playbook.yml --skip-tags deploy


# Pass extra variables at runtime

ansible-playbook playbook.yml -e 'nginx_port=8080'

ansible-playbook playbook.yml -e @vars.yml


# Start from a specific task

ansible-playbook playbook.yml --start-at-task='Deploy index.html'


# Syntax check only
```

```
ansible-playbook playbook.yml --syntax-check


# List all tasks without running

ansible-playbook playbook.yml --list-tasks
```

Quick Reference — ansible-playbook Options

- -i FILE : use this inventory file
- --check : dry run (show what would change, don't change)
- -v / -vvv : verbose / very verbose output
- --limit GROUP : only run on these hosts/groups
- --tags TAG : only run tasks with this tag
- --skip-tags T : skip tasks with this tag
- -e 'key=val' : pass extra variables
- --syntax-check: validate YAML syntax before running
- --list-tasks : print all tasks that would run
- --step : confirm each task before executing

Quick Reference Card

```
# ■■ INVENTORY ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ansible-inventory --list

ansible-inventory --graph

ansible all --list-hosts


# ■■ AD-HOC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ansible all -m ping

ansible all -m command -a 'uptime'

ansible webservers -b -m apt -a 'name=nginx state=present'

ansible all -b -m service -a 'name=nginx state=restarted'

ansible all -m setup

ansible all -m copy -a 'src=file dest=/remote/file'


# ■■ PLAYBOOK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ansible-playbook playbook.yml

ansible-playbook playbook.yml --check

ansible-playbook playbook.yml --limit webservers

ansible-playbook playbook.yml -e 'var=value'

ansible-playbook playbook.yml --syntax-check

ansible-playbook playbook.yml --tags deploy


# ■■ USEFUL ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

ansible --version
```

```
ansible-doc -l # list all modules

ansible-doc apt # docs for apt module

ansible-galaxy install role # install a role from Galaxy
```